



Práctica 5:

ANÁLISIS DE SOBREAJUSTE Y SUBAJUSTE MEDIANTE REGRESIÓN KVECINOS MÁS CERCANOS KNN EN PYTHON

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: 6CM3

Nombres de los integrantes:

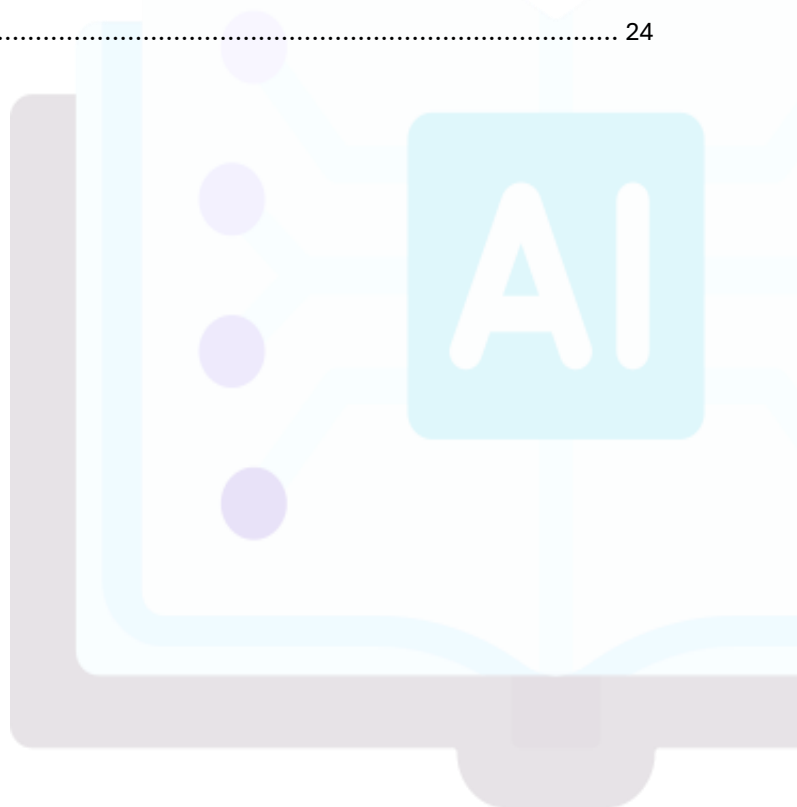
- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Notebook: [Practica no.5](#)

Miércoles, 15 de octubre, 2025

CONTENIDO

MARCO TEÓRICO	3
PROCEDIMIENTO	5
CARGA DEL ARCHIVO CARROS.CSV EN PYTHON	5
VISUALIZACIÓN DE LOS DATOS EN UNA GRÁFICA DE DISPERSIÓN	5
DIVISIÓN DEL CONJUNTO DE DATOS EN ENTRENAMIENTO Y PRUEBA	6
ENTRENAMIENTO DEL MODELO KNN PARA DISTINTOS VALORES DE K	7
GRÁFICA DE LOS RESULTADOS OBTENIDOS PARA CADA VALOR DE K	8
PERSONALIZACIÓN DE COLORES Y ESTILOS DE LAS GRÁFICAS	12
RESULTADOS	14
CONCLUSIONES	21
AGUIRRE LANTO VÍCTOR MANUEL	23
GASCA FRAGOSO PEDRO	23
GUEVARA BADILLO ARELI ALEJANDRA	23
MONTIEL TORO ARAEL DE JESÚS	24
RAMÍREZ LOZANO GAELE MARTÍN	24



MARCO TEÓRICO

SOBREAJUSTE vs. SUBAJUSTE en los Algoritmos de Machine Learning

En el mundo de Machine Learning, el objetivo principal es crear modelos que puedan hacer predicciones precisas sobre datos nuevos y nunca antes vistos. Para lograrlo, es fundamental encontrar un punto de equilibrio entre dos problemas muy comunes: el sobreajuste (overfitting) y subajuste (underfitting). Entender la diferencia entre ambos es clave para construir modelos de aprendizaje automático que sean realmente útiles y fiables.

En esencia, el sobreajuste y el subajuste representan los dos extremos de cómo un modelo puede aprender de los datos. Mientras que el sobreajuste es como una camisa a medida, ajustada perfectamente a un solo cuerpo, pero si alguien nuevo la intenta usar lo más probable es que no le quede bien, mientras que el subajuste es como una camisa de talla media la cual le puede quedar grande a algunas personas y ajustada a otras.

Características	SOBREAJUSTE	SUBAJUSTE	MODELO IDEAL
Rendimiento en Entrenamiento	Muy Alto	Bajo	Bueno
Rendimiento en Prueba	Bajo	Bajo	Bueno
Complejidad del Modelo	Demasiado Complejo	Demasiado Simple	Equilibrada
Capacidad de Generalización	Pobre	Pobre	Alta
Analogía de la Camisa	Solo le queda a la persona que se ajusto	A algunas les queda ajustadas y a otras holgadas	Una camisa que se adapte según el cuerpo

El objetivo ideal es alcanzar un "buen ajuste" (Good Fit). Un modelo bien ajustado es aquel que ha aprendido los patrones generales de datos sin memorizar el ruido. Es lo suficientemente complejo para capturar la esencia del problema, pero no tanto como para volverse inflexible.

EJEMPLO DE CLASIFICACIÓN CON KNN

El algoritmo K-Nearest Neighbors es un método de clasificación simple e intuitivo que clasifica un nuevo punto de datos basándose en la mayoría de votos de sus k vecinos más cercanos. El valor de k (el número de vecinos a considerar) es crucial.

Conjunto de Datos

Tenemos un conjunto de datos de 10 pacientes y queremos predecir si tienen una enfermedad (Clase 1) o no (Clase 0) basándonos en dos biomarcadores (Biomarcador A y Biomarcador B).

paciente	biomarcador A	biomarcador B	clase
1	2	3	0
2	3	4	0
3	4	2	0
4	5	5	0
5	9	8	1
6	10	9	1
7	11	7	1
8	12	10	1
9	6	6	1
10	5	8	0

Supongamos que llega un nuevo paciente (nuevo_P) con valores:

- Biomarcador A = 7
- Biomarcador B = 7

Queremos clasificarlo.

Caso 1: SOBREAJUSTE ($k=1$)

Paciente nuevo: nuevo_P (7,7)

Vecino más cercano: Paciente 9 (6,6)

Predicción: Clase 1 (enfermedad)

La frontera de decisión se vuelve muy irregular y específica para los datos de entrenamiento.

El modelo es demasiado específico y ha aprendido el ruido.

Caso 2: SUBAJUSTE ($k=9$)

Paciente nuevo: nuevo_P (7,7)

Se deben de tomar los 9 vecinos más cercanos, pero tenemos 10 puntos, por lo que la proporción similar o idéntica a la del dataset.

PROCEDIMIENTO

Carga del archivo carros.csv en Python

Se importan las bibliotecas necesarias y se carga el archivo carros.csv que contiene la base de datos con los kilómetros recorridos y el precio de los automóviles.

La librería panda se emplea para manipular la información en formato de tabla (*DataFrame*).

Se usa la función `files.upload()` para seleccionar el archivo directamente desde el equipo, algo ideal en Google Colab.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()

carros = pd.read_csv('carros.csv')

carros.head()
```

Visualización de los datos en una gráfica de dispersión

En este paso se busca comprender visualmente la relación entre las dos variables del conjunto de datos: los kilómetros recorridos por el vehículo (kms) y su precio (precio).

Se utiliza la función `scatter()` del módulo Matplotlib, que representa cada registro como un punto en el plano cartesiano:

- El eje X muestra los kilómetros recorridos.
- El eje Y representa el precio en pesos mexicanos.

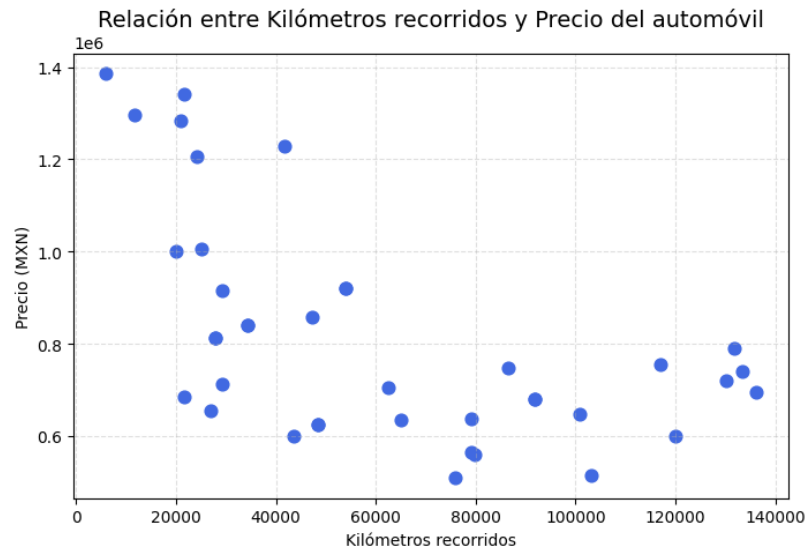
Este tipo de gráfico es muy útil en regresión porque permite detectar patrones o tendencias; en este caso, se espera observar una relación inversa (los precios disminuyen conforme aumentan los kilómetros).

También se agregan etiquetas, título y una cuadrícula para mejorar la legibilidad.

```
plt.figure(figsize=(8,5))
plt.scatter(carros["kms"], carros["precio"], color="royalblue", s=60)
plt.title("Relación entre Kilómetros recorridos y Precio del automóvil", fontsize=14)
plt.xlabel("Kilómetros recorridos")
plt.ylabel("Precio (MXN)")
plt.grid(True, linestyle="--", alpha=0.4)
plt.show()
```

La gráfica muestra una nube de puntos donde los precios más altos se concentran en vehículos con pocos kilómetros, mientras que los precios tienden a bajar conforme el kilometraje aumenta.

Este comportamiento confirma la existencia de una tendencia decreciente, que el modelo KNN intentará aprender en los pasos posteriores.



División del conjunto de datos en entrenamiento y prueba

En este paso se realiza la división del conjunto de datos en dos subconjuntos:

- Entrenamiento (80 %): se utiliza para ajustar o entrenar el modelo.
- Prueba (20 %): se reserva para evaluar el desempeño del modelo con datos que no ha visto.

Esta separación es fundamental en el aprendizaje automático, pues permite evaluar la capacidad de generalización del modelo, es decir, qué tan bien puede predecir sobre nuevos datos.

Para lograrlo, se emplea la función `train_test_split()` del módulo `sklearn.model_selection`.

El parámetro `random_state=42` garantiza que la partición sea reproducible cada vez que se ejecute el código.

El resultado son cuatro subconjuntos:

- `X_train` → variable independiente (kilómetros) para entrenamiento
- `X_test` → variable independiente para prueba
- `y_train` → variable dependiente (precio) para entrenamiento
- `y_test` → variable dependiente para prueba

```
from sklearn.model_selection import train_test_split

# Variable independiente (kms) y dependiente (precio)
X = carros[["kms"]]
y = carros["precio"]

# División 80% - 20%
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Mostrar tamaños resultantes
print("Tamaño del conjunto de entrenamiento:", len(X_train))
print("Tamaño del conjunto de prueba:", len(X_test))
```

El resultado muestra la cantidad de registros que fueron asignados a cada subconjunto tras aplicar la función `train_test_split()`.

En este caso, del total de 39 observaciones contenidas en el archivo `carros.csv`, 31 registros (aproximadamente el 80 %) se destinaron al entrenamiento del modelo, mientras que 8 registros (el 20 %) se reservaron para la fase de prueba.

Esta distribución confirma que los datos se dividieron de forma correcta y balanceada, lo que permitirá entrenar al modelo KNN con la mayoría de los datos y posteriormente evaluar su capacidad de generalización con un conjunto independiente.

Tamaño del conjunto de entrenamiento: 31

Tamaño del conjunto de prueba: 8

Entrenamiento del modelo KNN para distintos valores de K

En este paso implementamos el modelo de regresión K-Nearest Neighbors (KNN), es una técnica que se usa para estimar el valor de una variable continua en función de la cercanía con los valores más próximos en el conjunto de entrenamiento, en este caso lo haremos para el precio del automóvil

El algoritmo KNN se basa en un principio: elementos similares presentan comportamientos similares, es decir, para predecir el precio de un carro, el modelo considera los precios de sus K vecinos más cercanos y calcula un promedio ponderado

Antes del entrenamiento, los datos se escalan mediante una clase llamada `MinMaxScaler`, que transforma los valores al rango [0, 1]. Esto es muy importante porque KNN utiliza distancias euclidianas para medir la similitud, si las escalas son muy diferentes, una variable puede dominar a la otra.

Posteriormente se definen distintos valores de K (1, 3, 5, 10, 15, 20) para analizar como va a cambiar el comportamiento del modelo en cada caso.

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsRegressor

# Escalado de los datos (normalización al rango 0-1)
scaler_x = MinMaxScaler()
scaler_y = MinMaxScaler()

# Ajustar el escalador con los datos de entrenamiento
X_train_scaled = scaler_x.fit_transform(X_train)
y_train_scaled = scaler_y.fit_transform(y_train.values.reshape(-1, 1))

# Definición de los valores de K a evaluar
valores_k = [1, 3, 5, 10, 15, 20]
print("Valores de K definidos para el modelo:", valores_k)
```

La salida mostrará la lista de valores de K que se evaluarán.

En cada uno de ellos, el modelo KNN tomará un número distinto de vecinos para calcular la predicción del precio.

- K pequeño (ej. 1 o 3): el modelo se ajustará mucho al ruido → puede provocar sobreajuste.
- K grande (ej. 20 o más): el modelo será más general y perderá detalle → puede causar subajuste.

Valores de K definidos para el modelo: [1, 3, 5, 10, 15, 20]

Gráfica de los resultados obtenidos para cada valor de K

En este paso se entrena el modelo KNN con diferentes valores de K definidos previamente y se grafican los resultados para cada caso.

Cada iteración del bucle crea un modelo con un valor distinto de K, lo entrena con los datos normalizados (X_train_scaled, y_train_scaled) y luego genera una línea de predicción sobre el rango completo de kilómetros recorridos.

En la gráfica:

- Los puntos morados representan los datos reales (kms vs precio).
- La línea rosa muestra las predicciones del modelo para ese valor de K.

Estas gráficas permiten visualizar cómo cambia la complejidad del modelo:

- Cuando K es pequeño, la línea se ajusta demasiado a los puntos (*sobreajuste*).
- Cuando K es grande, la línea se suaviza demasiado (*subajuste*).

```
import numpy as np
import matplotlib.pyplot as plt

# Crear un rango de valores de kilómetros para graficar la predicción continua
kms_rango = np.linspace(X["kms"].min(), X["kms"].max(), 300).reshape(-1, 1)

# Entrenar y graficar un modelo KNN para cada valor de K
for k in valores_k:
    modelo = KNeighborsRegressor(n_neighbors=k)
    modelo.fit(X_train_scaled, y_train_scaled)

    # Generar predicciones
    kms_rango_scaled = scaler_x.transform(kms_rango)
    pred_scaled = modelo.predict(kms_rango_scaled)
    pred = scaler_y.inverse_transform(pred_scaled)

    # Crear gráfica
    plt.figure(figsize=(6,4))
    plt.scatter(X, y, color="mediumorchid", s=50, label="Datos reales")
    plt.plot(kms_rango, pred, color="deeppink", linewidth=3, label=f"K = {k}")

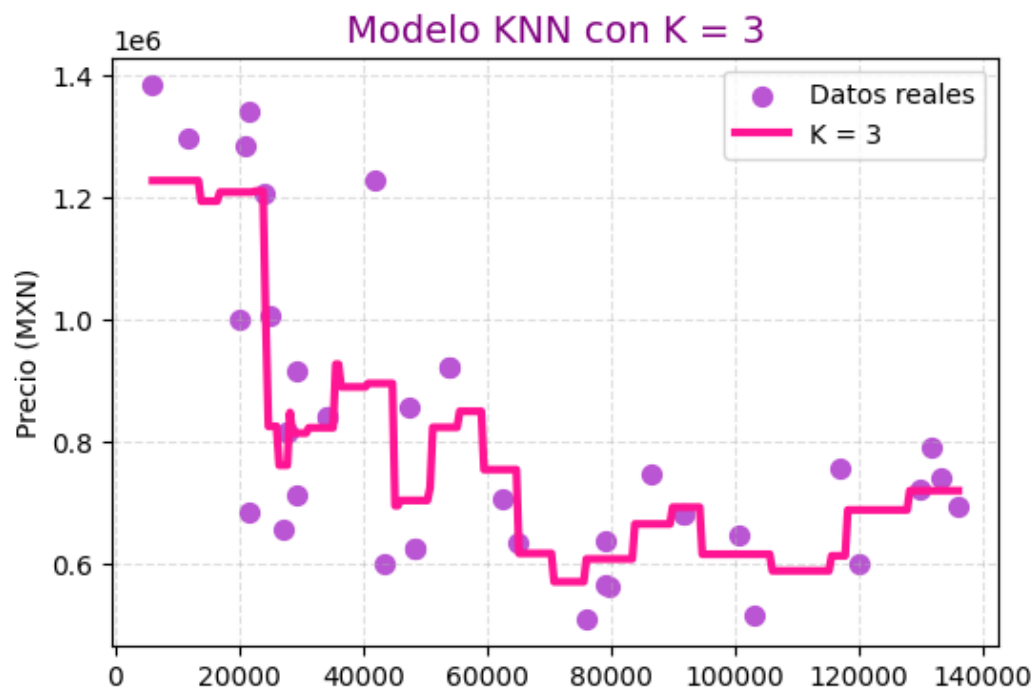
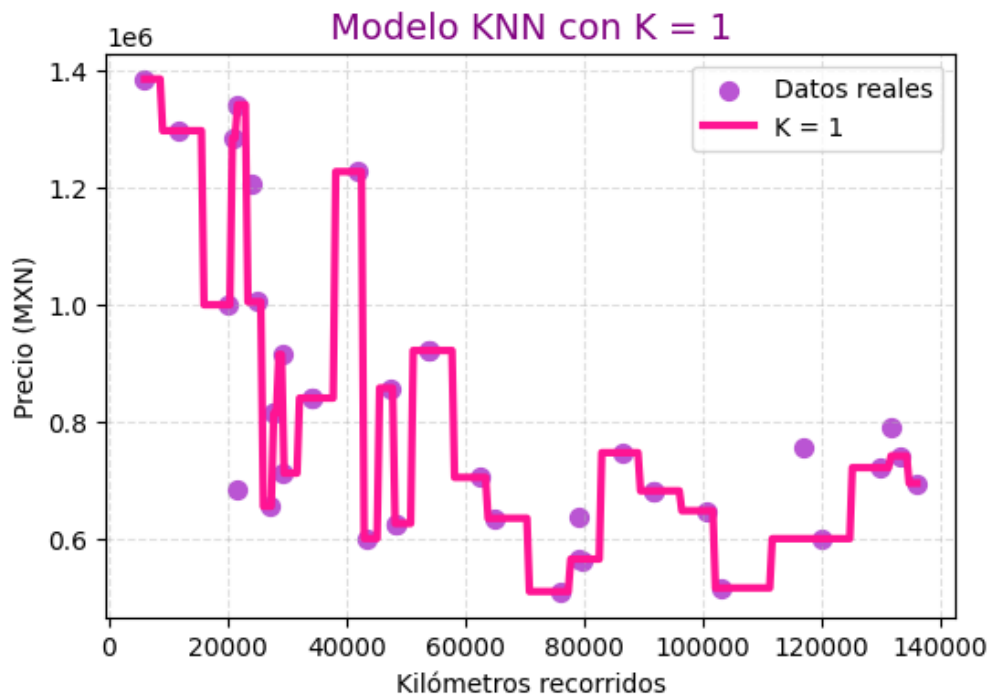
    plt.title(f"Modelo KNN con K = {k}", fontsize=14, color="purple")
    plt.xlabel("Kilómetros recorridos")
    plt.ylabel("Precio (MXN)")
    plt.legend()
    plt.grid(True, linestyle="--", alpha=0.4)
    plt.show()
```

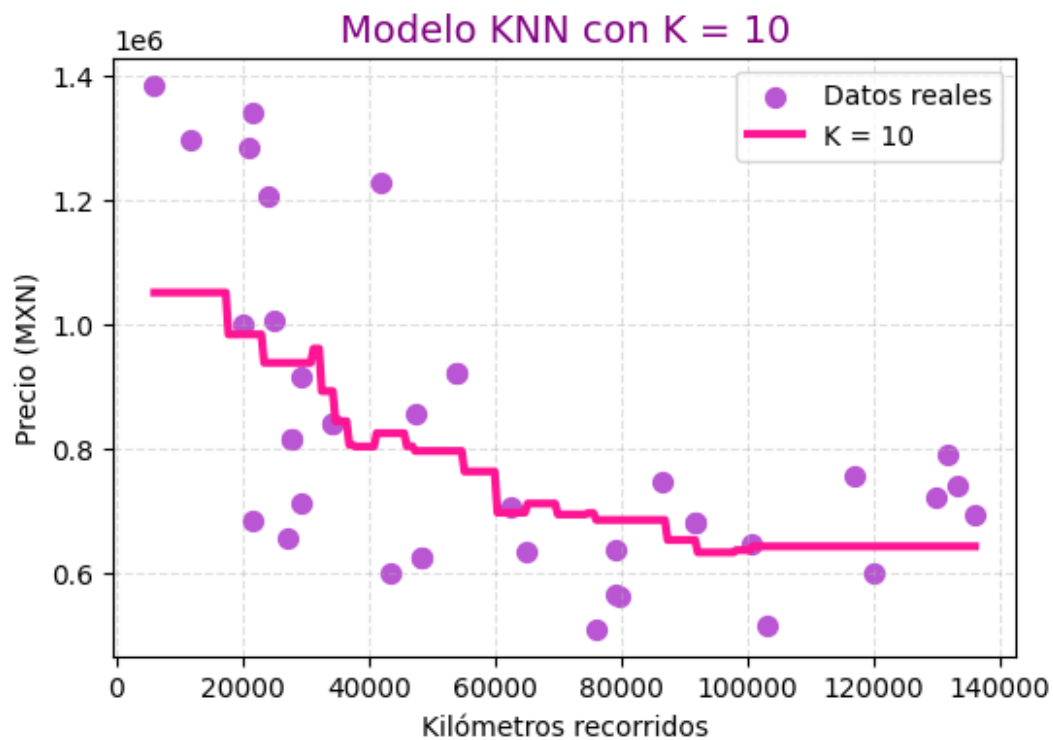
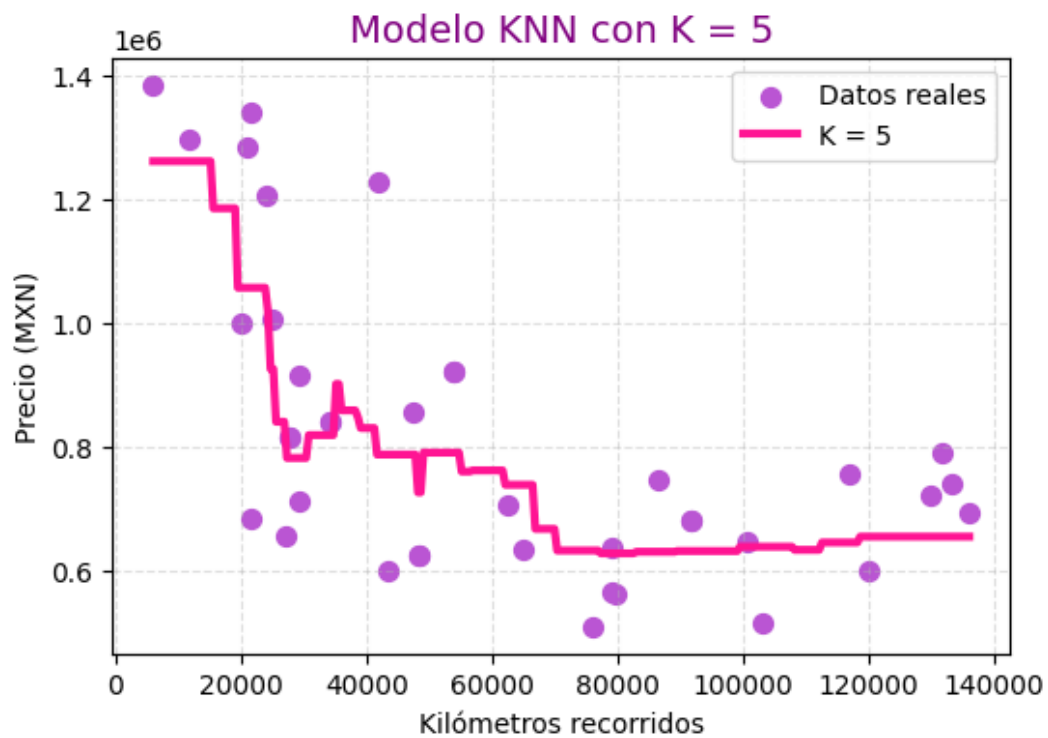
Al ejecutar el código del Paso 5, se generan varias gráficas, una por cada valor de K utilizado en el modelo KNN.

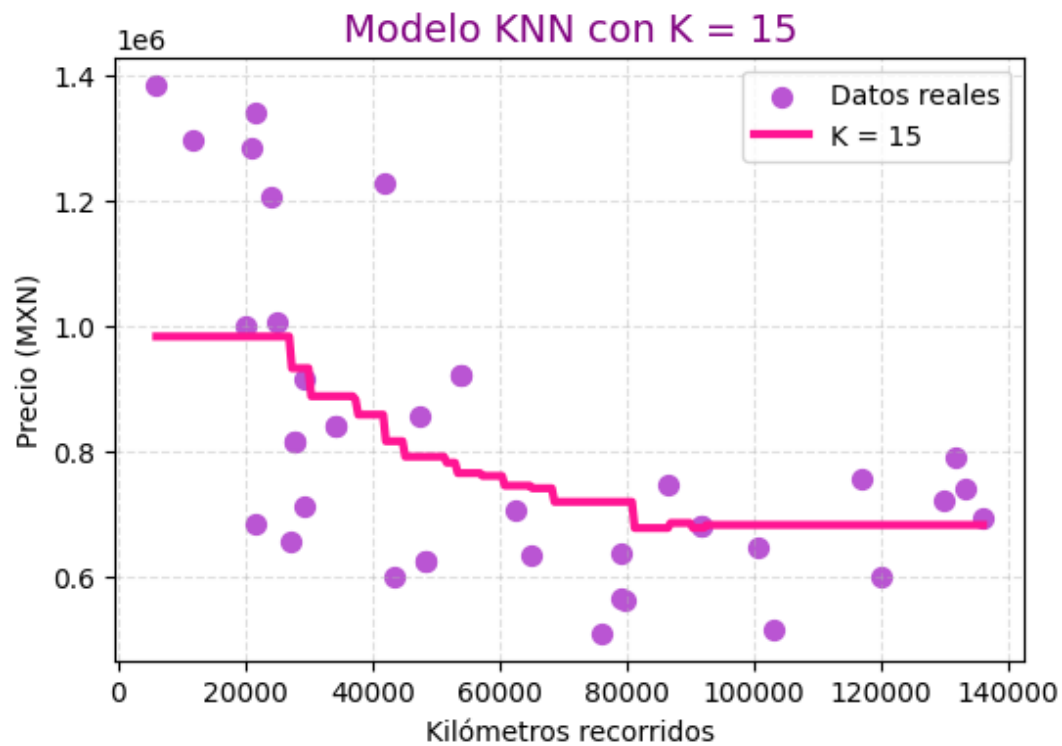
Cada figura permite visualizar cómo cambia el ajuste del modelo respecto a los datos reales:

- Para K = 1, la línea rosa sigue muy de cerca los puntos morados, adaptándose incluso a pequeñas variaciones o ruido.
Este comportamiento indica un *sobreajuste*, ya que el modelo se ajusta demasiado a los datos de entrenamiento y pierde capacidad de generalización.

- Conforme K aumenta ($K = 3, 5, 10, 15$), la curva se vuelve más suave y muestra una tendencia más estable.
En este rango, el modelo logra un equilibrio entre precisión y generalización.
- Para valores grandes de K ($K = 20$ o más), la línea casi se vuelve recta.
En este caso, el modelo se simplifica demasiado y no logra representar la variación real de los datos, lo cual se conoce como subajuste.







Personalización de colores y estilos de las gráficas

En este paso se realiza la personalización estética de las gráficas generadas.

El objetivo es mejorar la legibilidad y presentación visual de los resultados, utilizando distintos colores, estilos de líneas y temas de fondo.

La biblioteca Matplotlib permite modificar parámetros como:

- El color de los puntos (color= o c=).
- El color y grosor de la línea de predicción (linewidth=).
- El estilo de la cuadrícula y fondo mediante plt.style.use().

Estos ajustes no afectan los cálculos del modelo, pero hacen que las gráficas sean más claras y profesionales.

```
# Aplicar un estilo de fondo general
plt.style.use('seaborn-v0_8-darkgrid')

# Colores personalizados
color_puntos = "turquoise" # Color de los puntos (datos reales)
color_linea = "hotpink"    # Color de la línea del modelo

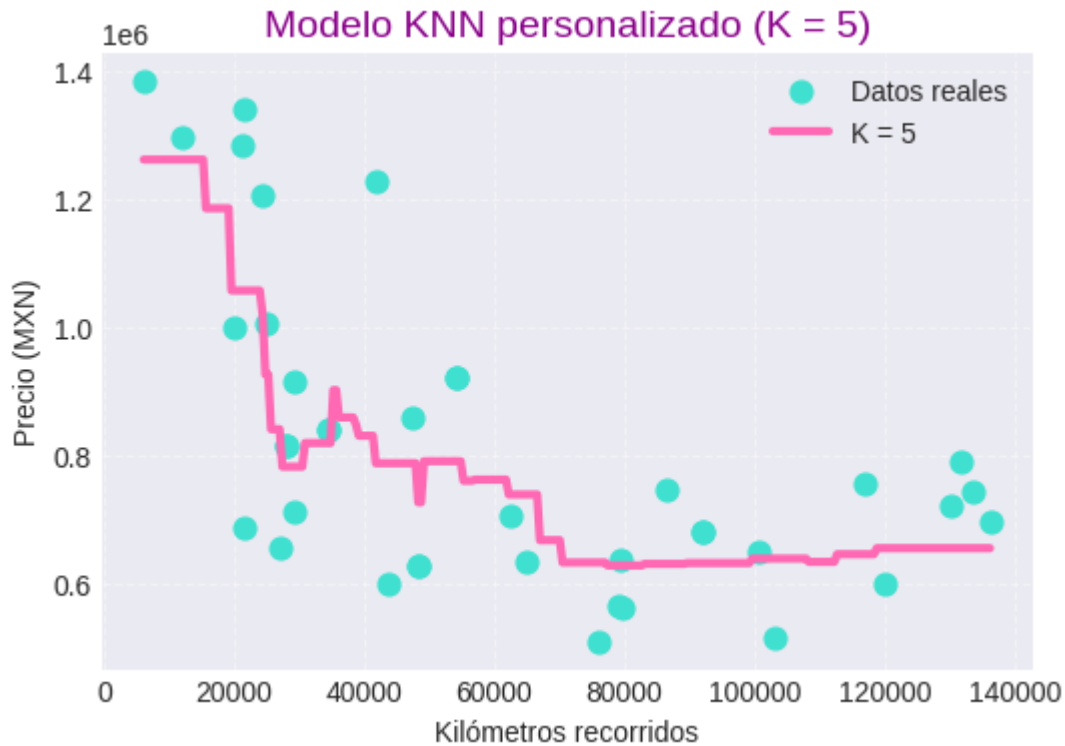
# Repetir la gráfica con los nuevos estilos
kms_rango = np.linspace(X["kms"].min(), X["kms"].max(), 300).reshape(-1, 1)

k = 5 # ejemplo con K=5, puedes cambiarlo
modelo = KNeighborsRegressor(n_neighbors=k)
modelo.fit(X_train_scaled, y_train_scaled)

kms_rango_scaled = scaler_x.transform(kms_rango)
pred_scaled = modelo.predict(kms_rango_scaled)
pred = scaler_y.inverse_transform(pred_scaled)

plt.figure(figsize=(6,4))
plt.scatter(X, y, color=color_puntos, s=60, label="Datos reales")
plt.plot(kms_rango, pred, color=color_linea, linewidth=3, label=f"K = {k}")
plt.title(f"Modelo KNN personalizado (K = {k})", fontsize=14, color="darkmagenta")
plt.xlabel("Kilómetros recorridos")
plt.ylabel("Precio (MXN)")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.4)
plt.show()
```

La nueva gráfica muestra los mismos resultados del modelo, pero con un esquema de colores personalizado y un estilo visual más agradable.



Una vez generadas todas las gráficas del Paso 5 (y personalizadas en el Paso 6), se procede a interpretar los resultados observados para los diferentes valores de K.

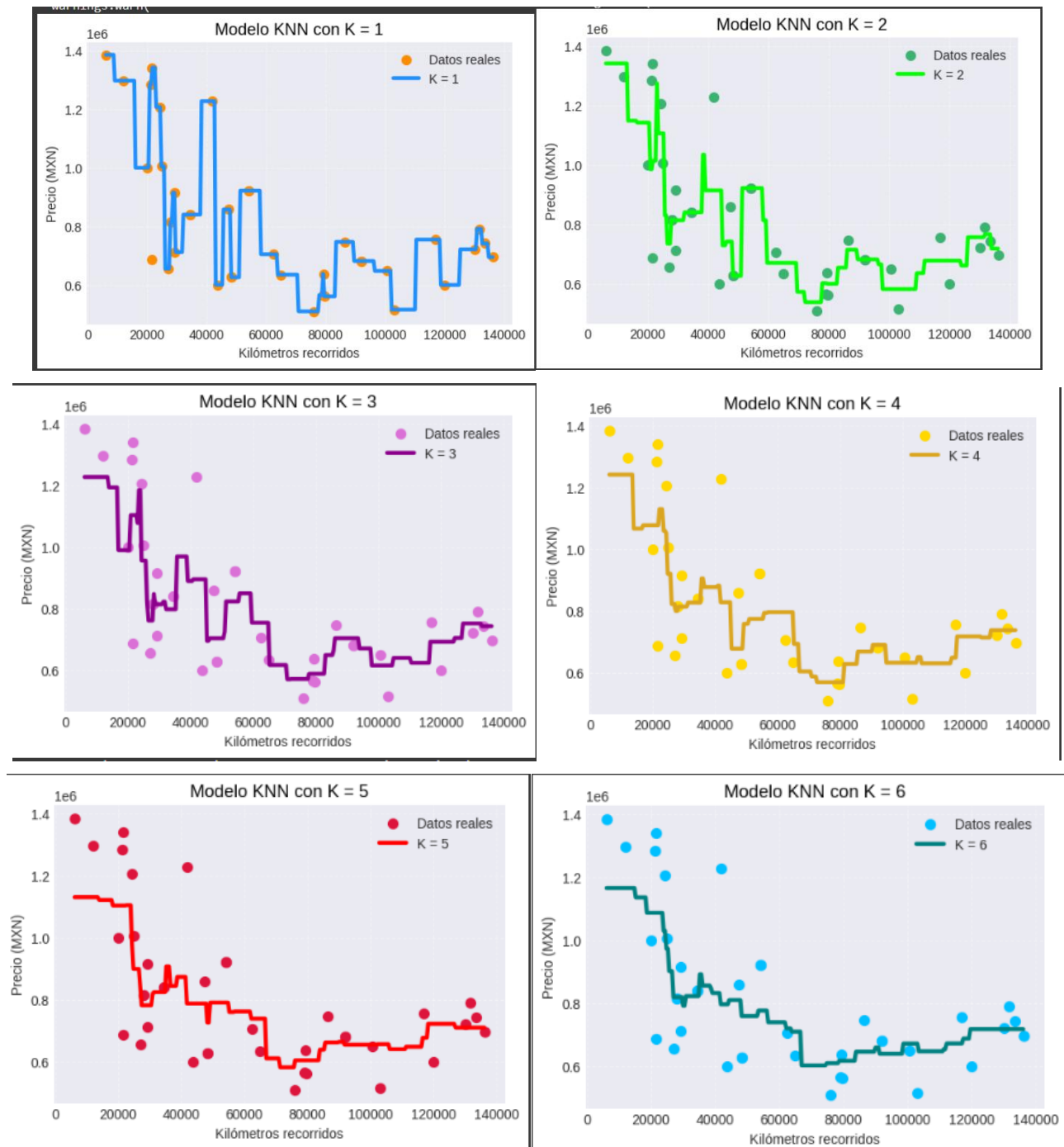
El objetivo es identificar los casos en los que el modelo presenta:

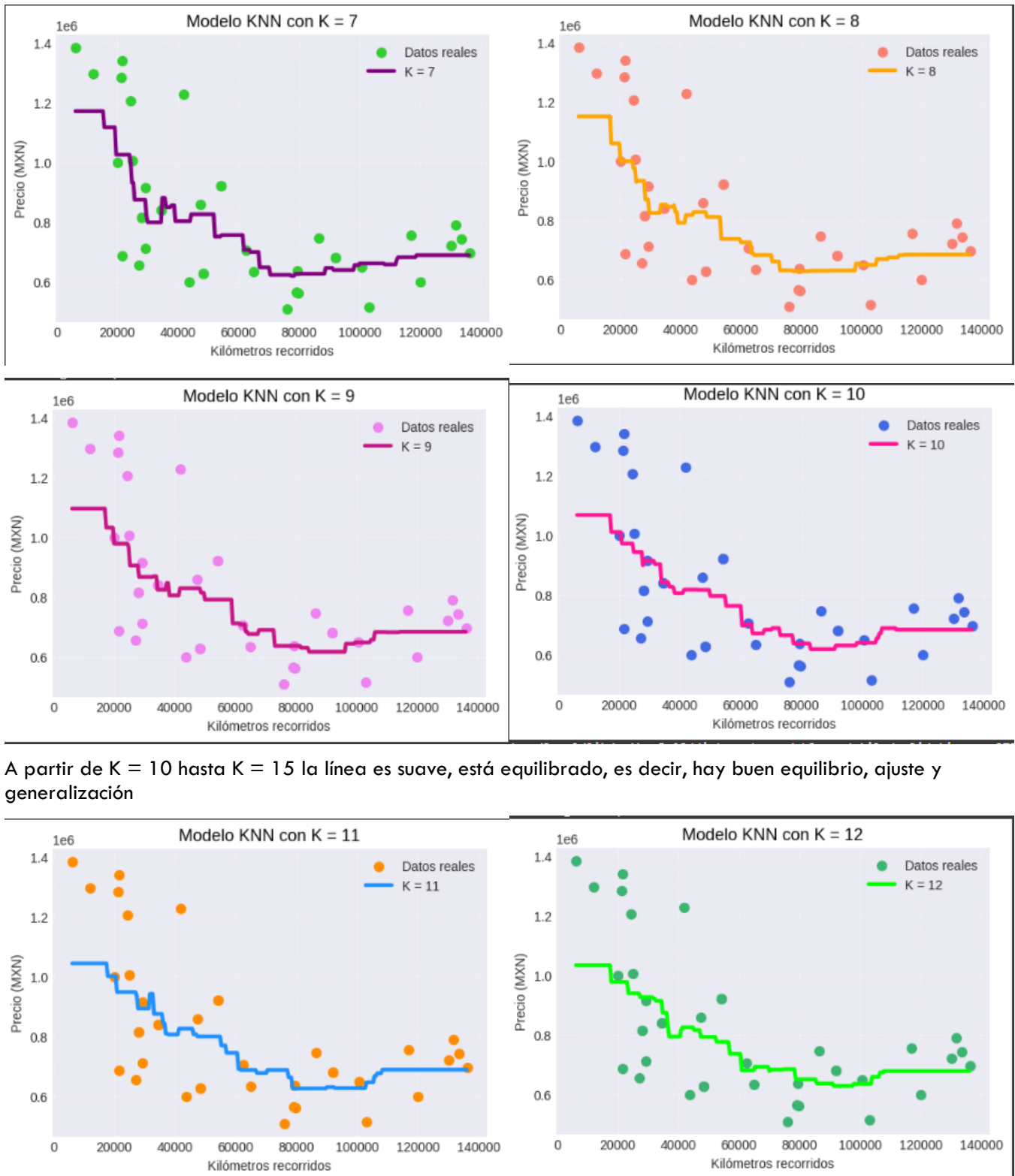
- **Sobreajuste (overfitting):**
 El modelo se ajusta demasiado a los datos de entrenamiento, siguiendo incluso el ruido.
 Ocurre cuando K es muy pequeño (por ejemplo, K = 1 o 3).
 En las gráficas, esto se observa cuando la línea rosa zigzaguea y pasa casi exactamente por cada punto.
- **Subajuste (underfitting):**
 El modelo es demasiado simple y no logra capturar la tendencia real de los datos.
 Sucede cuando K es muy grande (por ejemplo, K = 20 o más).
 En las gráficas, la línea aparece casi recta, sin adaptarse bien a las variaciones de los datos.

Resultados

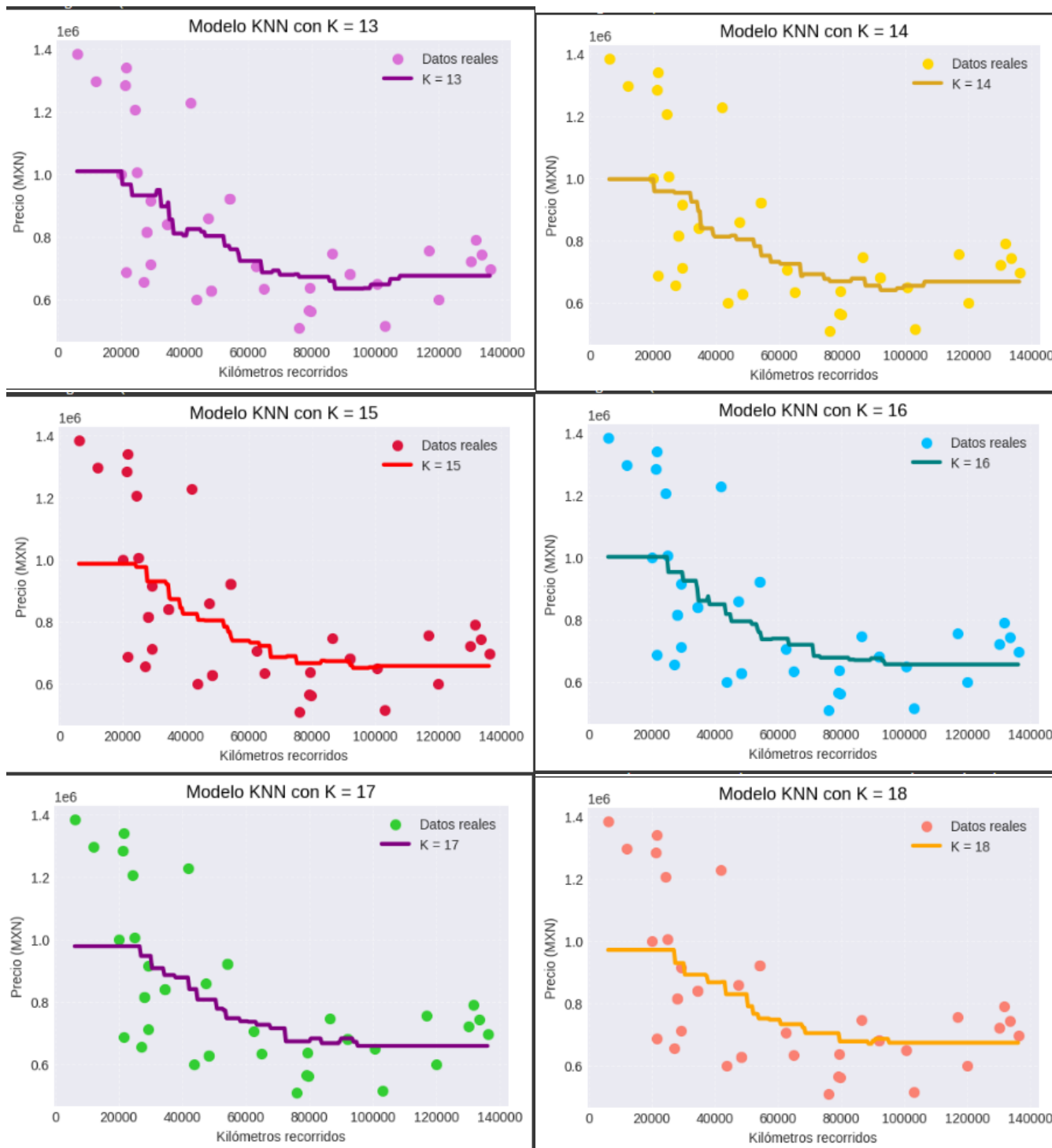
A continuación, se presentan todas las gráficas con $K = 1$ hasta $K = 39$

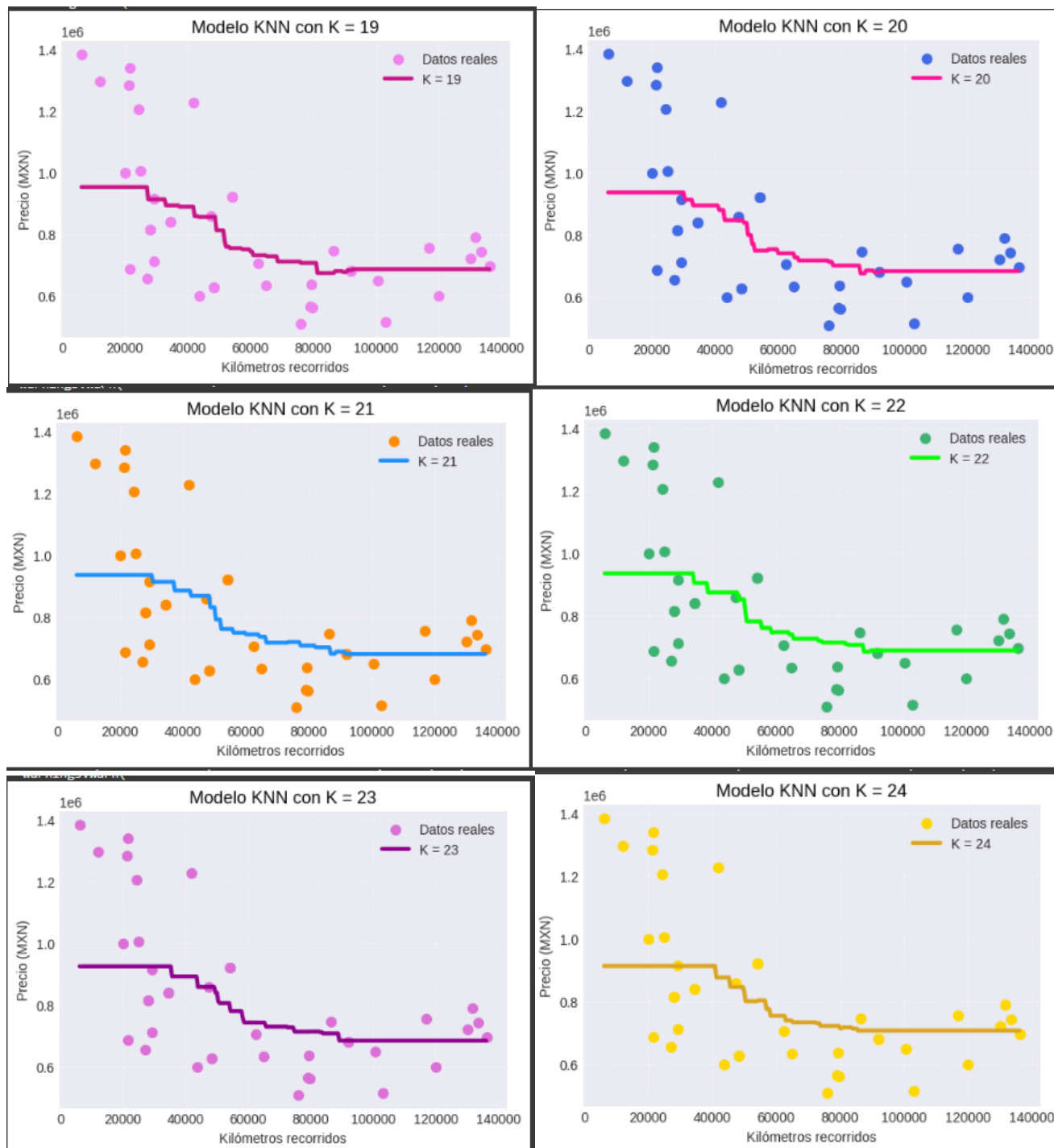
En las gráficas de $K = 1$ hasta $K = 3$ podemos ver mucho ruido, es una línea que sube y baja entre puntos, es decir, se observa un sobreajuste, es decir, el modelo está demasiado ajustado a los datos de entrenamiento, puede predecir bien con los datos de entrenamiento, pero tener problemas con nuevos datos



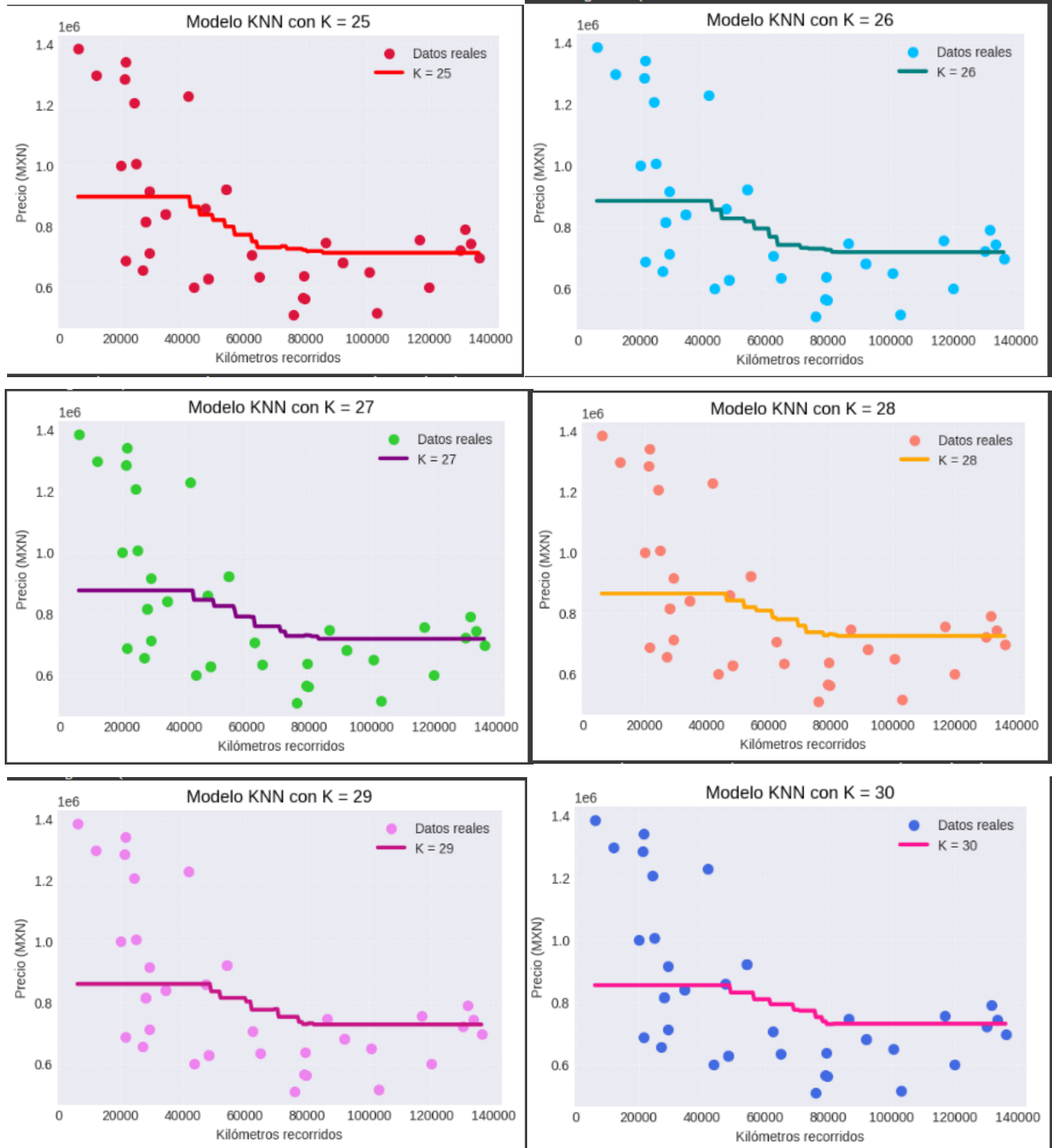


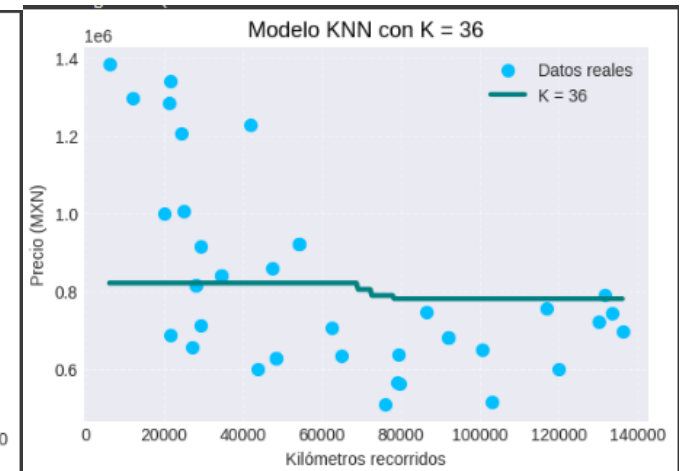
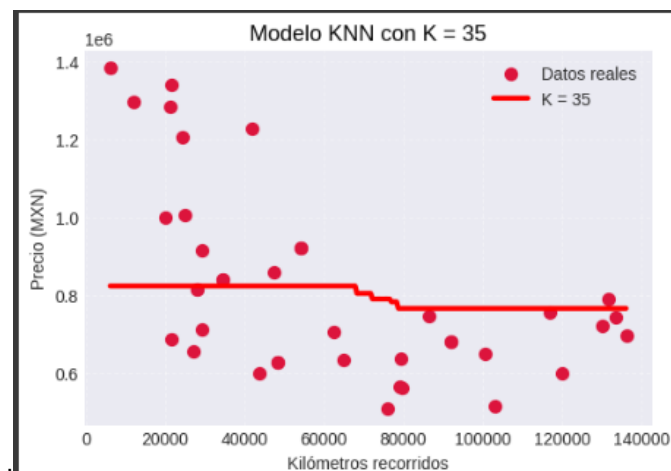
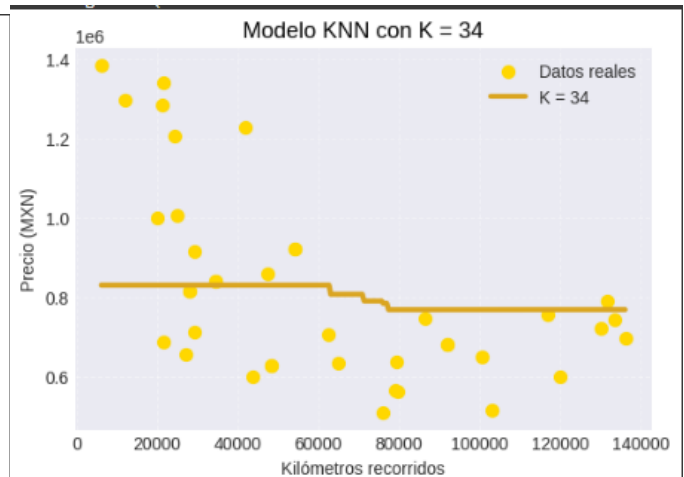
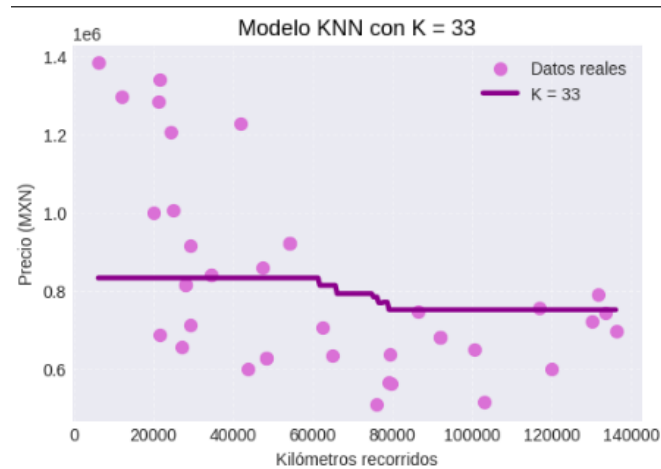
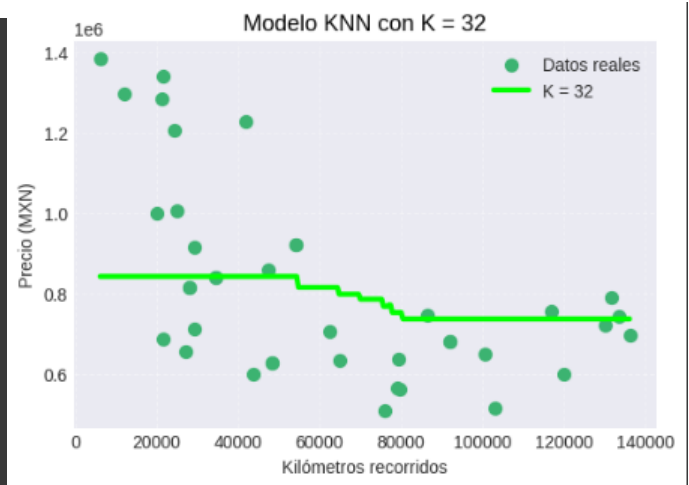
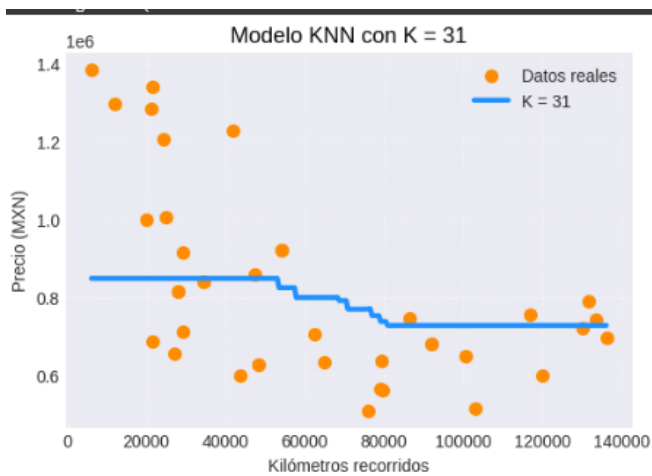
A partir de $K = 10$ hasta $K = 15$ la línea es suave, está equilibrado, es decir, hay buen equilibrio, ajuste y generalización

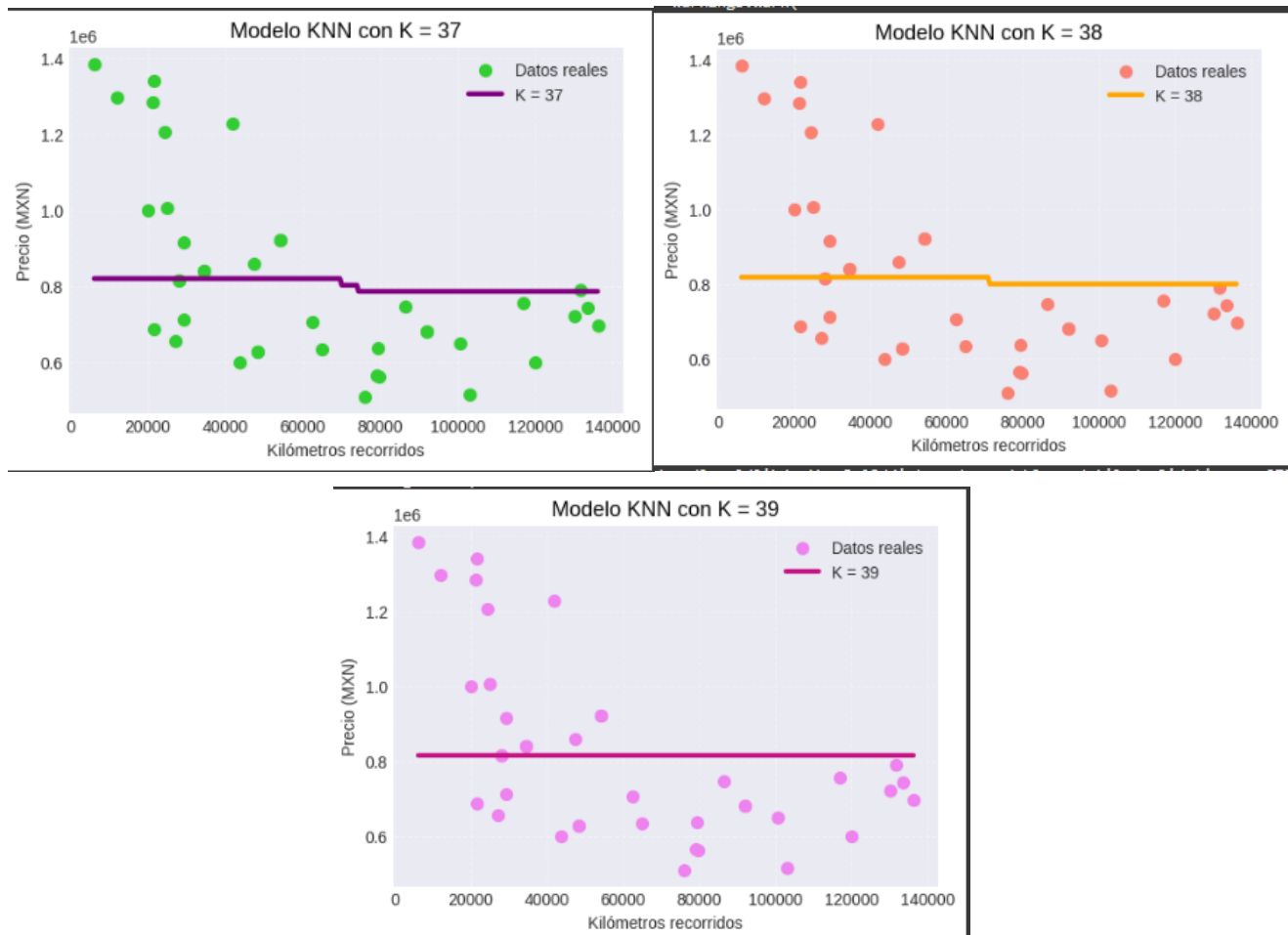




A partir de $K = 25$ empezamos a ver un subajuste, es decir, la línea es casi recta, muy rígida por lo que, el modelo es demasiado simple y presenta problemas para captar el patrón, es decir, no logra captar la relación real entre las variables y tiene errores tanto en entrenamiento como en prueba.







CONCLUSIONES

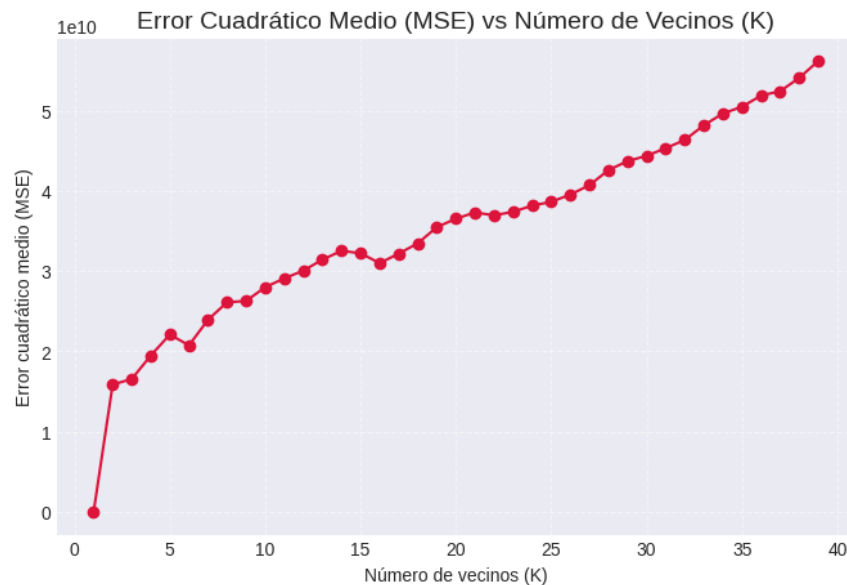
Error Cuadrático Medio

Código para calcular el MSE.

```
1 # Importar la función para calcular el error cuadrático medio
2 from sklearn.metrics import mean_squared_error
3
4 # Arreglo para guardar los errores
5 errores_mse = []
6
7 for num_vecinos in range(1, max_num_vecinos + 1):
8     # Crear y entrenar el modelo KNN
9     knn = KNeighborsRegressor(n_neighbors=num_vecinos)
10    knn.fit scaler_x.transform(X), scaler_y.transform(y.values.reshape(-1, 1)))
11
12    # Predicciones sobre los datos originales (para calcular el MSE)
13    y_pred_scaled = knn.predict(scaler_x.transform(X))
14    y_pred = scaler_y.inverse_transform(y_pred_scaled)
15
16    # Calcular el Error Cuadrático Medio (MSE)
17    mse = mean_squared_error(y, y_pred)
18
19    # Guardar el MSE en la lista
20    errores_mse.append(mse)
21
22 # Mostrar el arreglo de errores
23 print("Errores cuadráticos medios para cada K:")
24 for k, error in enumerate(errores_mse, start=1):
25     print(f"K = {k}: MSE = {error:.4f}")
26
27 # Graficar el error cuadrático medio vs número de vecinos
28 plt.figure(figsize=(8,5))
29 plt.plot(range(1, max_num_vecinos + 1), errores_mse, marker='o', linestyle='-', color='crimson')
30 plt.title("Error Cuadrático Medio (MSE) vs Número de Vecinos (K)", fontsize=14)
31 plt.xlabel("Número de vecinos (K)")
32 plt.ylabel("Error cuadrático medio (MSE)")
33 plt.grid(True, linestyle="--", alpha=0.5)
34 plt.show()
```

Tabla de Resultados

K	MSE	K	MSE	K	MSE
1	0	14	3.25E+10	27	4.07E+10
2	1.59E+10	15	3.22E+10	28	4.26E+10
3	1.66E+10	16	3.10E+10	29	4.37E+10
4	1.95E+10	17	3.22E+10	30	4.44E+10
5	2.21E+10	18	3.35E+10	31	4.53E+10
6	2.08E+10	19	3.55E+10	32	4.63E+10
7	2.40E+10	20	3.65E+10	33	4.82E+10
8	2.61E+10	21	3.73E+10	34	4.96E+10
9	2.63E+10	22	3.69E+10	35	5.05E+10
10	2.80E+10	23	3.74E+10	36	5.19E+10
11	2.91E+10	24	3.82E+10	37	5.24E+10
12	3.01E+10	25	3.86E+10	38	5.41E+10
13	3.14E+10	26	3.95E+10	39	5.62E+10



El análisis del Error Cuadrático Medio (MSE) revela el comportamiento del modelo KNN al variar el hiperparámetro K sobre los datos de entrenamiento.

- **K=1 (Sobreajuste Extremo):** Se obtuvo un **MSE de 0.00**, lo que indica un sobreajuste (overfitting) perfecto. El modelo ha memorizado los datos de entrenamiento en lugar de aprender el patrón general, por lo que no será útil para predecir nuevos datos.
- **Tendencia General (Hacia el Subajuste):** A medida que K aumenta, el MSE también crece de forma consistente. Esto muestra que el modelo se vuelve progresivamente más simple y pierde capacidad para capturar la relación real en los datos, tendiendo al subajuste (underfitting) o un modelo con alto sesgo.

Aguirre Lanto Víctor Manuel

Lo más revelador para mí fue encontrar el punto de equilibrio. Mientras que K valía uno sobreajustaba el modelo y valores como veinte o más lo subajustaban. Un rango intermedio, aproximadamente entre K igual a diez y quince, parecía ofrecer el mejor ajuste. En las gráficas de estos valores, la curva es suave y captura la tendencia general de que el precio baja a medida que aumentan los kilómetros, pero sin las fluctuaciones erráticas del sobreajuste. Aprendí que la relación entre la complejidad y la generalización no es lineal, sino que existe un punto dulce. El objetivo no es hacer el modelo lo más simple o complejo posible, sino encontrar el nivel de complejidad adecuado que le permita capturar el patrón subyacente en los datos, ignorando el ruido.

Gasca Fragoso Pedro

Min principal conclusión se centra en el subajuste, que fue evidente con valores de K grandes, específicamente a partir de $K=25$. En las gráficas correspondientes, se observa como la línea de predicción se vuelve cada vez más rígida y simplista, casi una línea recta que no logra capturar la relación decreciente real entre los kilómetros y el precio. Este fenómeno demuestra que un modelo demasiado simple, tampoco es bueno generalizando, ya que ignora patrones importantes y la varianza en los datos. La lección para mí es que la simplicidad extrema es tan perjudicial como la complejidad extrema; el modelo se vuelve tan general que no es específico para nada, resultando en errores altos tanto en los datos de entrenamiento como en los de prueba.

Guevara Badillo Areli Alejandra

Esta práctica me permitió usar las gráficas como una herramienta de diagnóstico fundamental. El sobreajuste con $K=1$ y el subajuste con $K > 20$ se manifiestan visualmente de maneras opuestas: el primero como una línea caótica y zigzagueante y el segundo como una línea excesivamente plana. Esto me demostró que la complejidad del modelo (controlada por K) debe ser cuidadosamente calibrada. Un modelo muy complejo (K bajo) tiene alta varianza y se adapta al ruido, lo que le impide generalizar. Por otro lado, un modelo poco complejo (K alto) tiene un alto sesgo y no captura la tendencia, lo que también le impide generalizar correctamente. La clave está en balancear este compromiso entre sesgo y varianza para construir un modelo que sea robusto y útil en escenarios reales.

Montiel Toro Arael de Jesús

En esta práctica, identifiqué claramente que un valor de $K=1$ produjo un sobreajuste extremo. Esto se refleja en la gráfica de $K=1$, donde la línea de predicción es muy irregular y zigzagueante, intentando pasar exactamente por cada uno de los puntos de datos de entrenamiento. Es como si el modelo hubiera "memorizado" los datos en lugar de aprender la tendencia general. Lo que aprendí es que un modelo excesivamente complejo (como un KNN con un solo vecino) pierde por completo su capacidad de generalización. Al ajustarse hasta al más mínimo ruido de los datos de entrenamiento, se vuelve inútil para predecir nuevos datos que no ha visto antes, ya que cualquier pequeña variación lo desviará por completo.

Ramírez Lozano Gael Martin

A través de esta práctica, comprendí la importancia de las métricas cuantitativas para evaluar el ajuste del modelo. El Error Cuadrático Medio (MSE) demostró numéricamente lo que las gráficas mostraban visualmente: $K=1$ tuvo un MSE de cero, un signo inequívoco de sobreajuste perfecto, mientras que el MSE crecía consistentemente a medida que K aumentaba, indicando una tendencia hacia el subajuste. La gráfica final del MSE contra K es la lección más importante para mí. Ilustra perfectamente que a medida que la complejidad del modelo disminuye (K aumenta), el error en los datos de entrenamiento aumenta. Esto demuestra que un modelo que es perfecto con los datos que ya conoce probablemente no generalizará bien, y que debemos aceptar un cierto nivel de error para crear un modelo que sea verdaderamente predictivo.